Scalable Conversational RAG with Guardrails

Emad Noorizadeh
Bank of America

October 19, 2025

Abstract

This white paper presents a scalable Retrieval-Augmented Generation (RAG) system for conversational workloads with production-grade guardrails. The system combines a robust ingestion and indexing pipeline, a stateful LangGraph orchestration layer, a guarded prompting scheme with strict JSON validation and evidence checks, explicit out-of-domain (OOD) refusal policies, and rich observability. We document the architecture, execution flow, guardrail mechanisms, and metrics. We also summarize our experimental attempts to introduce clarifying questions and why for now we recommend intent classification to govern behavior for vague or broad queries, while preserving a safe fallback to full-collection retrieval.

1 Introduction

Enterprises need conversational assistants that *answer grounded questions* from a controlled corpus while refusing unsupported claims and surfacing clear provenance. Our system targets:

- **Reliability**: reject or abstain when retrieval is weak or citations are invalid.
- **Safety**: never leak facts not present in the grounding context.
- **Scalability**: efficient ingestion, indexing, and retrieval across growing corpora.
- **Observability**: end-to-end metrics and logs to explain decisions.

2 Conversational RAG Flow

2.1 Index

1. **Chunking**: sentence-level splits with metadata.
2. **Embedding**: dense vectors.
3. **Persistence**: vectors in `ChromaDB` with parsed metadata in `index/`.
4. **Startup audit**: verify embedding dimensionality; backup/reset mismatched collections to keep retrieval aligned with the active embedding model.

2.2 Session

Session state retains:

- `last_question`, `awaiting_clarification`, `clarify_count`, `topic_hint`
- The full message transcript for multi-turn reasoning and follow-ups.

2.3 LangGraph Execution

The graph (`backend/router_graph.py`) executes:

1. **ingest**: append user turn; ensure persistent defaults.
2. **frustration**: scan utterances for tokens ("confusing", "hard", ...). If triggered, synthesize a friendly clarification prompt and short-circuit the rest.
3. **build_query**: form `effective_question` by stitching prior interpreted question to short fragments; reuse questions inside clarification loops; mark whether history was appended.
4. **retrieve**: call `RetrievalService.retrieve` with `RetrievalConfig` (`top_k`, `similarity_threshold`); cache per-chunk score vectors, `avg_similarity`, and any retrieval error on the RAG instance.
5. **answer**: assemble a 3-turn conversation snippet + `topic_hint`; construct the guarded prompt (`get_rag_main_prompt`); call `RAG.generate_response`; store the raw structured payload (`rag_response`).
6. **decide**: inspect `rag_response.metrics`. If `abstained=true` and a `clarifying_question` exists, switch to clarification mode (increment `clarify_count`); otherwise, classify answer vs. bare abstain; propagate `answer_text`, `interpreted_question`, decision, and clarification flags.
7. **finalize**: merge via `ChatAgent._create_intelligent_metrics`; fuse LLM metrics with retrieval stats (scores, chunk IDs, `route_decision`, `route_metrics.above_threshold`); build sources; stamp timestamps; update persistent fields; carry over `fallback_reason`; mark responder (`generated_by = answer_llm, router_guard, retrieval_error_handler, clarification_gua` ...).

3 Retrieval Layer

Inputs. `effective_question`, `RetrievalConfig{ top_k, similarity_threshold }`.

Outputs. Top-k semantic hits (text + metadata + scores), `avg_similarity`, and cached score vectors.

Integrity checks. On index startup, the Chroma config audit aligns dimensionality with the current embedding model and backs up or resets stale collections to avoid silent mismatch errors.

4 Guarded Generation and Evidence Discipline

4.1 Prompt and Schema

The answer node formats chunks as `C1: ...`, `C2: ...` and uses a guarded schema prompt. The LLM must return strict JSON with fields for *answer*, *evidence IDs*, *faithfulness/completeness scores*, *interpreted question*, and abstention metadata.

4.2 Validation Pipeline

1. Parse with `_parse_json_response`; validate with `_validate_response_schema`.
2. Verify cited evidence IDs exist; if any citation is invalid, force abstain (we never present unsupported grounding).
3. If the LLM abstains repeatedly despite high similarity, mutate the prompt (*"model abstained despite sufficient grounding context"*) and retry up to `models.llm_max_retry`. If still failing, emit fallback with `fallback_reason="abstained_after_retry"` and log the excerpt.
4. If the model client is missing, produce *model unavailable* abstain; any exception becomes `fallback_reason="generation_exception"`.

5 Guardrails and Routing Decisions

5.1 Out-of-Domain (OOD) and Weak Retrieval

The retrieve node records `avg_similarity` and sets `route_metrics.above_threshold`. If `avg_similarity` misses `chat_agent.similarity_threshold` (default 0.45), `finalize` labels the response **router_guard** and abstains with a clear message.

5.2 LLM Refusals with Sufficient Context

If the model explicitly abstains with valid context and no clarifying question, we mark **answer_guard**: a deliberate refusal with sufficient grounding.

5.3 Clarification Loops and Frustration Guard

Clarification loops preserve context by injecting the last question into follow-ups until the user answers or the clarification budget (`chat_agent.max_clarify`) is exhausted. Frustration short-circuits ensure negative statements do not consume retrieval/LLM cycles.

6 Metrics and Analytics

6.1 Context Utilization and Alignment

`calculate_context_utilization_percentage` synthesizes:

- **Grounded precision/recall**: proportion of answer tokens supported by retrieved chunks and coverage of salient chunk content.

- **Numeric alignment**: consistency of numbers vs. sources.
- **Unsupported terms**: residual tokens not grounded.
- **Semantic similarity**: MiniLM cosine similarity between Q and A.
- **Entity coverage**: optional spaCy NER extraction to assess whether key entities are addressed.

These metrics are fused in `finalize` with route decisions and guardrail labels so the UI debug panel mirrors backend behavior.

6.2 Logging for Traceability

`backend.router_graph.router` and `backend.rag.rag` log each node with timings, decisions, retrieval stats, fallback reasons, and answer lengths. Combined with metrics, operators can diagnose why a turn produced `answer_guard`, `router_guard`, or `retrieval_error_handler`.

7 Reliability, Performance, and Scalability

Embedding audits. On startup, collections that mismatch the active embedding model dimension are backed up/reset, preventing silent retrieval skew after model upgrades.

Latency and cost. The graph avoids unnecessary LLM calls via router and frustration guards; retries are bounded by `models.request_timeout` and `models.llm_max_retry`.

Scalability. Ingestion and indexing are streaming-friendly; retrieval operates over ChromaDB with metadata filters available for future sharding or intent-scoped collections.

8 Experiments: Clarification Strategies (Summary)

We tested two families to introduce clarifying questions without disrupting UX:

LLM policy in the prompt. We asked the model to propose a clarifying question when a faithful answer would exceed a word budget and require multiple facets. *Outcome:* The model rarely clarified (preference to answer); coupling to abstain meant broad-but-answerable prompts did not trigger; length prediction was weak.

Deterministic dispersion gate. We computed rank-weighted document probabilities $p(d)$ over top-K hits, entropy $H_{\text{doc}} = -\sum_d p(d) \ln p(d)$, perplexity $E_{\text{doc}} = \exp(H_{\text{doc}})$, and predicted length $\widehat{L} = \mu \cdot E_{\text{doc}}$ (online EMA of words-per-mode). *Outcome:* With K=5, E_{doc} under-triggered; label noise (boilerplate) yielded poor options; the calibrator drifted when updated on abstains; no post-LLM fallback led to abstain instead of a clarifying question.

Conclusion. Clarification adds maintenance and requires deeper retrieval instrumentation (clean labels, $K \geq 30$, stable parent IDs) to be robust. For now, we defer.

9 Roadmap: Intent Classifier to Govern RAG

We will address broad and vague queries via an **intent classifier**:

- **Taxonomy:** e.g., Checking, Savings, Cards, Loans, Preferred Rewards, Fees, Eligibility, Overview/All, OOD.
- **Training:** bootstrap with silver labels from logs (query $\rightarrow$ clicked family); expand with LLM-assisted labeling and review.
- **Routing:** if intent confidence $\geq \tau$, retrieve from intent-scoped collections; else fall back to the full collection.
- **Vague/OOD:** low confidence yields suggestions/chips or abstain; user may explicitly request "Overview/All" to scan the full collection.

This keeps behavior deterministic, reduces coupling to generation prompts, and doubles as a retrieval accelerator.

10 Configuration Reference (Excerpt)

Key	Default	Description
`models.embedding_model`	(env)	Embedding model for indexing/retrieval.
`chat_agent.similarity_threshold`	0.45	Retrieval sufficiency threshold for router guard.
`models.request_timeout`	(env)	Timeout for LLM calls.
`models.llm_max_retry`	1–2	Retry budget for repair/abstain-after-retry.
`metrics.use_spacy_ner`	true	Enable spaCy-backed entity extraction for context utilization.
`reports.final_reports_path`	`./reports/final`	Where PDFs appear in the UI.
`chat_agent.max_clarify`	1	Clarification loop budget.

11 Conclusion

This system delivers grounded conversational answers with explicit guardrails, robust indexing integrity checks, and full-stack observability. While clarification is valuable, our experiments show it requires additional retrieval instrumentation to avoid UX regressions. An intent classifier will provide the next layer of control for vague and broad prompts, enabling intent-scoped retrieval with safe fallbacks.